Stato	Completato
Iniziato	martedì, 1 settembre 2026, 10:14
Terminato	martedì, 1 settembre 2026, 10:53
Tempo impiegato	39 min. 18 secondi

Domanda 1

Risposta corretta

In C#, i tipi "primitivi" (es. `int`) sono...

- [ ] a. tipi speciali che non hanno alcun corrispondente nel CLR
- [ ] b. sempre tipi riferimento
- [x] c. alias di tipi del namespace `System` (es. `System.Int32`)
- [ ] d. indipendenti dal Common Type System

La risposta corretta è: alias di tipi del namespace `System` (es. `System.Int32`)

Domanda 2

Risposta corretta

Stai scrivendo un Dockerfile. Devi inserire i file del codice sorgente nell'immagine. Perché la documentazione e le best practice suggeriscono di usare `COPY . .` invece di `ADD . .` se non hai esigenze particolari?

- [ ] a. Perché `COPY` crea un layer più piccolo rispetto ad `ADD`
- [x] b. Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie
- [ ] c. Perché `ADD` funziona solo in modalità rootless
- [ ] d. Non c'è differenza, sono alias dello stesso comando

La risposta corretta è: Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie

Domanda 3

Risposta corretta

Quali delle seguenti differenze/relazioni tra `git fetch` e `git pull`, sono corrette (nella configurazione di default)?

- [ ] a. `git pull` non scarica mai nulla
- [x] b. `git fetch` scarica dati dal remoto ma non modifica la working directory
- [ ] c. `git fetch` scarica dati dal remoto e modifica, di conseguenza, la working directory
- [x] d. `git pull` (di default) equivale a fetch+merge
- [ ] e. `git fetch` non scarica mai nulla

Le risposte corrette sono: `git fetch` scarica dati dal remoto ma non modifica la working directory, `git pull` (di default) equivale a fetch+merge

Domanda 4

Risposta corretta

Perché esiste `IDisposable` in .NET?

- [ ] a. Ragioni storiche/di compatibilità: da C# 8 non serve più
- [x] b. Serve per rilasciare deterministicamente risorse non gestite che sono incapsulate da oggetti .NET
- [ ] c. Per permettere agli sviluppatori di creare nuove classi

La risposta corretta è: Serve per rilasciare deterministicamente risorse non gestite che sono incapsulate da oggetti .NET

Domanda 5

Risposta corretta

Esegui il comando `git switch main`. Cosa succede?

- [x] a. I file nella *Working Directory* vengono aggiornati per rispecchiare il contenuto dell'ultimo commit del branch `main`.
- [x] b. Il puntatore `HEAD` viene aggiornato per puntare al riferimento `refs/heads/main`
- [ ] c. Viene creato un nuovo commit di merge se c'erano modifiche non "committate"
- [ ] d. Se ci sono modifiche che vanno in conflitto con il branch `main`, Git le butta via

Le risposte corrette sono: I file nella *Working Directory* vengono aggiornati per rispecchiare il contenuto dell'ultimo commit del branch `main`., Il puntatore `HEAD` viene aggiornato per puntare al riferimento `refs/heads/main`

Domanda 6

Risposta corretta

`run:` vs `uses:` in un step di GitHub Actions: quali affermazioni sono corrette?

- [ ] a. `uses:` serve solo per stampare stringhe
- [x] b. `run:` esegue comandi di shell
- [x] c. `uses:` richiama una action (es. `actions/checkout@v4`)
- [ ] d. sono degli alias, si possono usare in modo intercambiabile
- [x] e. `run:` può essere usato per eseguire `dotnet test`

Le risposte corrette sono: `run:` esegue comandi di shell, `uses:` richiama una action (es. `actions/checkout@v4`), `run:` può essere usato per eseguire `dotnet test`

Domanda 7

Risposta corretta

Quali operazioni sono tipicamente possibili via reflection?

- [ ] a. Generare automaticamente unit-test
- [x] b. Creare istanze con `Activator.CreateInstance(...)`
- [x] c. Invocare metodi con `MethodInfo.Invoke(...)`
- [ ] d. Creare nuove istanze con `new`
- [x] e. Caricare assembly e scoprire tipi a runtime (plugin)

Le risposte corrette sono: Creare istanze con `Activator.CreateInstance(...)`, Invocare metodi con `MethodInfo.Invoke(...)`, Caricare assembly e scoprire tipi a runtime (plugin)

Domanda 8

Risposta corretta

Perché alcuni Dockerfile contengono più istruzioni `FROM`, per esempio:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
...
FROM mcr.microsoft.com/dotnet/runtime:9.0
COPY --from=build ...
```

Qual è lo scopo di questa tecnica?

- a. Velocizzare il download dell'immagine permettendo il download parallelo dei layer
- b. Ridurre la dimensione dell'immagine finale scartando gli strumenti di compilazione (SDK) e mantenendo solo il runtime e i binari compilati ✓
- c. Permettere al container di eseguire sia codice .NET 9 che codice legacy
- d. Creare due immagini distinte con un solo comando `docker build`

La risposta corretta è: Ridurre la dimensione dell'immagine finale scartando gli strumenti di compilazione (SDK) e mantenendo solo il runtime e i binari compilati

Fine revisione